Specifica dei Requisiti e Architettura del Sistema

Progetto **Locridea Transfer**

Documento di Analisi Tecnica

Versione 2.1 - Evoluzione Architetturale & UX Avanzata

Redatto da:
Alessandro Taverniti

Data:
6 aprile 2026

Indice

1 Introduzione **2**
1.1 Scopo del Sistema 2
1.2 Definizioni, Acronimi e Abbreviazioni 3

2 Requisiti **4**
2.1 Requisiti Funzionali (RF) 4
2.2 Requisiti Non Funzionali (RNF) 4

3 Diagramma dei Casi d'Uso **5**
3.1 Descrizione generale 5

4 Schede dei Casi d'Uso **6**
4.1 CU1: Prenotare Transfer NCC 6
4.2 CU2: Gestire Prenotazioni (Dashboard Admin) 10
4.3 CU3: Gestire Preventivi non confermati (Bozze) 13
4.4 CU4: Gestire Tariffe e parametri di calcolo 16

5 Modello di Dominio **19**
5.1 Descrizione del Modello 19

6 Diagrammi di Sequenza del Sistema (SSD) **20**
6.1 SSD CU1: Prenotare Transfer 20
6.2 SSD CU2: Gestire Prenotazioni 20
6.3 SSD CU3: Gestire Preventivi (Bozze) 20
6.4 SSD CU4: Gestire Tariffe 21

7 Architettura Logica e Infrastrutturale **22**
7.1 Design Pattern Implementati 22
7.2 FinOps e CI/CD Automation 22

1 Introduzione

1.1 Scopo del Sistema

L'obiettivo del sistema **Locridea Transfer** è fornire una piattaforma completa e automatizzata per la prenotazione e la gestione logistica di servizi NCC (Noleggio Con Conducente) in Calabria. Il sistema permette ai clienti finali di:

- Ottenere preventivi istantanei tramite integrazione con Google Maps e avere un'anteprima del tragitto.
- Prenotare **Pacchetti Turistici** a tariffa fissa predefinita.
- Completare una prenotazione guidata multi-step ad alta conversione, con recupero bozze.
- Ricevere conferma della prenotazione tramite WhatsApp il prima possibile.

Parallelamente, il sistema mette a disposizione una **Smart Dispatch Dashboard** (Admin) per la gestione operativa avanzata, che include:

- **Quick Actions Logistiche**: Integrazione deep-link per navigazione GPS nativa e salvataggio rapido a calendario degli incarichi.
- Visualizzazione delle prenotazioni con calcolo istantaneo del margine di profitto e dei chilometri a vuoto (Rimessa).
- Incrocio visivo dei dati logistici (es. orari voli/treni vs orario pick-up) per prevenire disservizi.
- Modifica dei prezzi, dei parametri tariffari e consultazione storico.

L'applicazione è progettata per servire principalmente due tipologie di utenti:

- **Cliente**: l'utente finale che richiede un transfer NCC.
- **Amministratore (Admin)**: il gestore dell'azienda che amministra il sistema gestionale protetto da autenticazione e Policy di Sicurezza (RLS).

1.2 Definizioni, Acronimi e Abbreviazioni

Di seguito sono riportate le definizioni dei termini tecnici e degli acronimi utilizzati nel documento.

Termine	**Descrizione**
Admin	Utente con privilegi di amministrazione del sistema (Back-office).
Deep Link / URI	Collegamento che scavalca il browser e invoca direttamente un'applicazione nativa (es. Google Maps o App Calendario).
Design Pattern	Soluzioni architetturali standard a problemi comuni di progettazione software (es. Strategy, Repository).
FinOps	Pratiche di ottimizzazione cloud per massimizzare il valore di business e ridurre i costi server.
Next.js	Framework React utilizzato per lo sviluppo dell'applicazione web (SSR e Server Actions).
Pacchetto Turistico	Tratta predefinita a tariffa fissa isolata dal motore di calcolo dinamico (bypass di Google Maps).
Quick Actions	Azioni rapide integrate nella UI per azzerare i tempi di data-entry manuale dell'autista.
RLS	*Row Level Security*: Meccanismo di sicurezza di PostgreSQL/Supabase per limitare l'accesso ai dati.
Rimessa (Dead Mileage)	Chilometraggio a vuoto che il mezzo deve percorrere per raggiungere il punto di partenza o per rientrare.
Strategy Pattern	Pattern comportamentale utilizzato per incapsulare diverse logiche di calcolo tariffario in classi intercambiabili.
Supabase	Piattaforma Backend-as-a-Service basata su PostgreSQL utilizzata per database e autenticazione.
Zod	Libreria TypeScript utilizzata per la validazione rigorosa e type-safe degli schemi dati.

2 Requisiti

2.1 Requisiti Funzionali (RF)

RF-1 **Flusso di Prenotazione Cliente**: Il sistema deve permettere al Cliente di accedere al form pubblico compilando dinamicamente i dati di viaggio (tratta custom) o selezionando un Pacchetto Turistico a tariffa fissa.

RF-2 **Motore di Pricing Dinamico**: Il sistema deve calcolare automaticamente il prezzo stimato integrando le API di Google Maps (server-side) con parametri tariffari (scaglioni km, maggiorazioni notturne, tempi di attesa e costo di *rimessa*).

RF-3 **Gestione Pacchetti Turistici**: Il sistema deve supportare prenotazioni di tratte predefinite bypassando il motore dinamico.

RF-4 **Login Admin**: Il sistema deve fornire un'area riservata protetta da middleware per l'autenticazione.

RF-5 **Smart Dispatch (Admin)**: L'Admin deve visualizzare le richieste, interagire con i dati tramite **Quick Actions** (apertura navigatore nativo, generazione evento in calendario), confermarle, rifiutarle o rinegoziarne il prezzo.

RF-6 **Storico e Audit**: Il sistema deve mantenere uno storico dei cambi di stato (Soft Delete) e un audit log immutabile.

RF-7 **Gestione Tariffe Base**: L'Admin deve poter aggiornare in autonomia i parametri di costo.

2.2 Requisiti Non Funzionali (RNF)

RNF-1 **Sicurezza Backend-Driven**: Il calcolo delle distanze e dei prezzi deve avvenire via Server Actions. Le rotte `/admin/*` devono essere protette da middleware.

RNF-2 **Protezione Dati (DB)**: I dati sensibili devono essere protetti tramite Row Level Security (RLS) su PostgreSQL.

RNF-3 **Type-Safety End-to-End**: L'intero stack deve essere tipizzato utilizzando TypeScript e validato tramite schemi Zod.

RNF-4 **Infrastruttura e FinOps**: Implementare pipeline CI/CD personalizzata (Deploy Hooks) per abbattere i costi di licenza server (Team Seats).

3 Diagramma dei Casi d'Uso

3.1 Descrizione generale

Il diagramma dei casi d'uso di Locridea Transfer descrive le interazioni tra il sistema e i due attori principali: il **Cliente** e l'**Amministratore**.

Il **Cliente** interagisce principalmente con il caso d'uso *Prenotare Transfer NCC* (CU1), che copre la ricerca del percorso, il calcolo del preventivo e la conferma della prenotazione.

L'**Amministratore** gestisce il Back-Office attraverso i casi d'uso operativi principali: *Gestire Prenotazioni* (CU2), *Gestire Preventivi non confermati* (CU3) e *Gestire Tariffe* (CU4). L'accesso a tutte le funzionalità amministrative è subordinato al *Login* nell'area riservata.

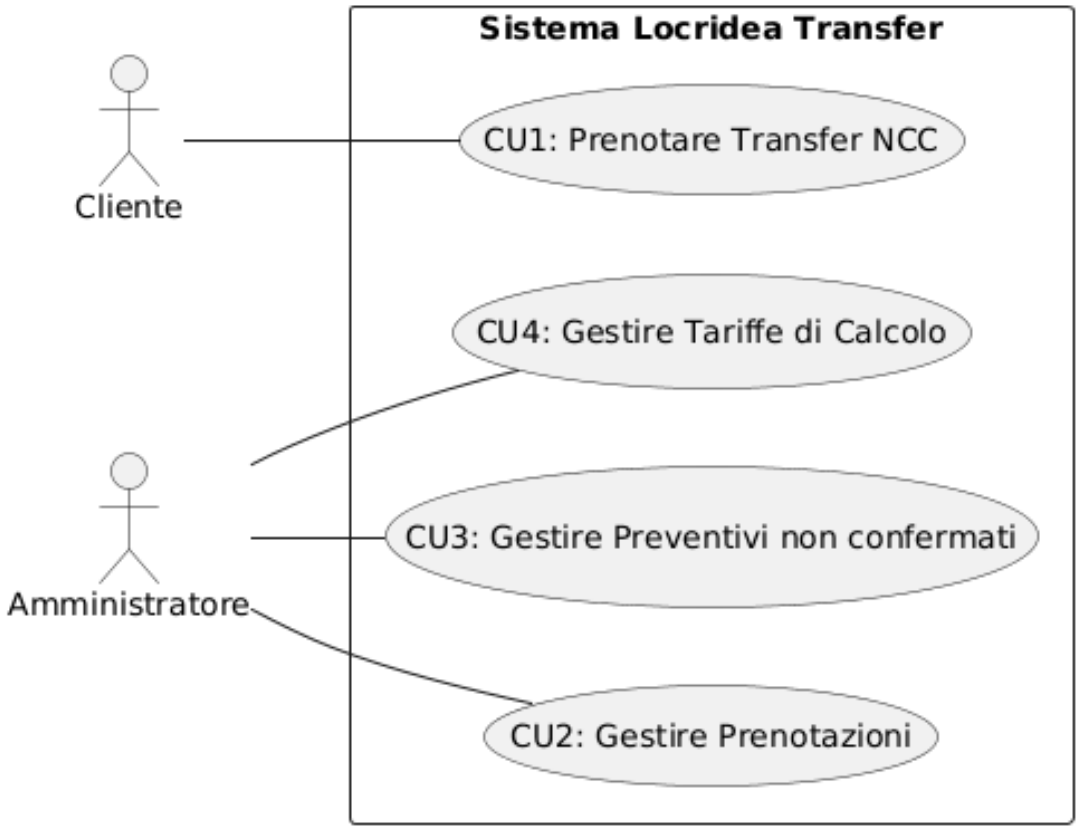

Figura 1: Diagramma dei Casi d'Uso del sistema

4 Schede dei Casi d'Uso

4.1 CU1: Prenotare Transfer NCC

Portata Sistema Locridea Transfer

Livello Obiettivo utente

Attore primario Cliente

Parti interessate ed interessi

- **Cliente**: Vuole prenotare un transfer in modo semplice, veloce e trasparente, con prezzo chiaro.
- **Azienda**: Vuole ricevere richieste consistenti (dati corretti, orari realistici) validate lato server.
- **Sistema**: Deve garantire correttezza dati, eseguire calcolo (Google Maps), prevenire spam e salvare stati (bozze).

Scenario principale di successo

1. Il Cliente apre la homepage e seleziona la prenotazione transfer.
2. Inserisce **indirizzo di partenza** e **arrivo** (tramite Google Places Autocomplete).
3. Seleziona **data e ora** del viaggio.
4. Indica dettagli aggiuntivi (tipo transfer, note).
5. Inserisce i **dati passeggeri** (adulti, bambini, neonati e seggiolini).
6. Compila i **dati anagrafici** (nome, cognome, telefono).
7. Visualizza e accetta l'informativa privacy.
8. Il Sistema invoca il **calcolo del preventivo** (Google Maps + Pricing Engine).
9. Il Cliente visualizza il riepilogo con anteprima MAPS e prezzo stimato.
10. Il Cliente conferma la prenotazione ("Conferma").
11. Il Sistema applica controlli anti-abuso e valida nuovamente i dati.
12. Il Sistema crea il record nella tabella `bookings` con stato `DA CONFERMARE`.
13. Il Sistema mostra la pagina di successo.

Estensioni e flussi alternativi

1b. Istanziazione da Pacchetto Turistico *(Flusso Parallelo)*
Il Cliente seleziona un "Pacchetto" predefinito dalla vetrina. Il Sistema **inibisce l'uso delle API di Google Maps** e del motore dinamico. La tariffa viene agganciata al valore fisso del Pacchetto salvato a database. L'utente procede inserendo unicamente data, ora e anagrafica.

2a. Indirizzo non valido *(Passo 2)*
Il Sistema non ottiene risultati da Google Places, mostra errore e blocca l'avanzamento. Il Cliente corregge l'input.

2b. Contenuto non consentito negli indirizzi *(Passo 2)*
Il Cliente inserisce nell'indirizzo di partenza o di arrivo un testo che contiene termini non consentiti (blocklist). Il Sistema rileva la presenza di tali termini, blocca l'avanzamento e mostra il messaggio: "Il testo inserito contiene termini non consentiti. Per favore modifica i campi indicati." Il Cliente corregge il contenuto e riprova.

3a. Data/ora non valide *(Passo 3)*
Il Cliente seleziona data passata o orario non gestibile. Il Sistema richiede modifica.

6a. Dati anagrafici incompleti *(Passo 6)*
Il Cliente lascia vuoti uno o più campi obbligatori (nome, cognome, telefono). Il Sistema blocca l'avanzamento e richiede la compilazione di tutti i campi obbligatori.

6b. Contenuto non consentito in nome o cognome *(Passo 6)*
Il Cliente inserisce nel nome o nel cognome un testo che contiene termini non consentiti. Il Sistema blocca l'avanzamento e mostra: "Il testo inserito contiene termini non consentiti. Per favore modifica i campi indicati." Il Cliente modifica nome e/o cognome e riprova.

6c. Numero di telefono non valido *(Passo 6)*
Il Cliente inserisce un numero di telefono che non supera le verifiche (lunghezza, formato, cifre uguali). Il Sistema blocca l'avanzamento e mostra: "Inserisci un numero di telefono valido per il paese selezionato." Il Cliente corregge il numero e riprova.

6d. Email con dominio non consentito *(Passo 6)*
Il Cliente inserisce un'email il cui dominio è in blocklist (es. email temporanee). Il Sistema rifiuta la richiesta e restituisce: "Indirizzo email non consentito." Il Cliente modifica l'email o lascia il campo vuoto (se facoltativo).

8a. Errore Google Maps *(Passo 8)*
La chiamata fallisce. Il Sistema logga l'errore e informa l'utente ("Impossibile calcolare preventivo"). Se non previsto fallback manuale, il caso termina.

11a. Anti-bot / Rate limiting *(Passo 11)*
Rilevato comportamento anomalo (bot o troppe richieste). Il Sistema rifiuta la richiesta senza salvare.

11b. Contenuto non consentito nelle richieste speciali *(Passo 11)*
Il Cliente ha compilato il campo "Richieste speciali" con termini non consentiti. Il

Sistema, in fase di validazione, rifiuta l'invio e mostra l'errore. Il Cliente modifica o svuota il campo e riprova.

11c. Numero di telefono non valido in fase di invio *(Passo 11)*
Il Sistema verifica nuovamente il telefono in fase di conferma. Se non valido, rifiuta l'invio e mostra errore. Il Cliente torna allo step dati anagrafici per correggere.

11d. Totale passeggeri non valido *(Passo 11)*
Il Sistema verifica che la somma dei passeggeri non superi il massimo consentito. Se superato, restituisce errore e non salva. Il Cliente modifica il numero di passeggeri.

11e. Volo o treno obbligatorio mancante *(Passo 11)*
Per transfer "Aeroporto" o "Stazione" mancano i dettagli obbligatori (volo/treno). Il Sistema blocca l'invio e richiede i dati. Il Cliente torna allo step precedente e compila.

13a. Errore scrittura DB *(Passo 12-13)*
Timeout o errore interno su Supabase. Il Sistema mostra errore generico, il Cliente può riprovare.

14a. Uscita anticipata *(Qualsiasi passo)*
Il Cliente abbandona prima della conferma. I dati non vengono salvati come prenotazione confermata.

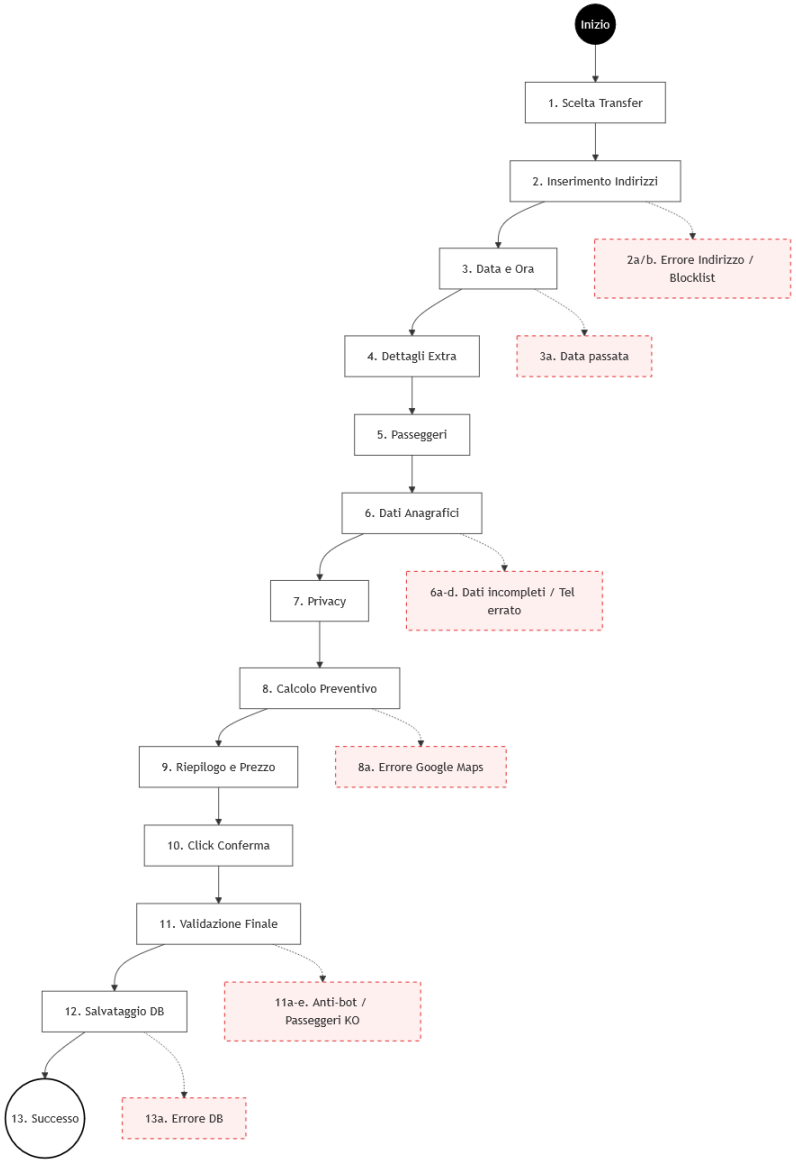

Figura 2: Diagramma di attività - CU1: Prenotare Transfer NCC

4.2 CU2: Gestire Prenotazioni (Dashboard Admin)

Portata Sistema Locridea Transfer

Livello Obiettivo utente

Attore primario Amministratore (Admin)

Parti interessate ed interessi

- **Admin**: Vuole una Control Room mobile-first per confermare/rifiutare richieste in base alla fattibilità logistica e supportato da calcoli su *rimessa* e profitto.
- **Azienda**: Vuole un processo sicuro, tracciato (audit log) e sicuro (niente hard delete).
- **Sistema**: Deve autorizzare solo gli Admin, gestire gli stati e registrare audit.

Interazioni UX e Quick Actions Operative

La card della singola prenotazione funge da *hub operativo mobile* per minimizzare il carico cognitivo dell'autista:

- **Click-to-Navigate (Deep Linking):** Cliccando sulla stringa dell'itinerario all'interno della card, il sistema genera un URI Scheme che apre automaticamente l'applicazione nativa Google Maps o Apple Maps con le coordinate di partenza e arrivo già preimpostate, azzerando i tempi di data-entry manuale.
- **Click-to-Calendar (.ics / Event Payload):** Cliccando sull'orario o sulla data della prenotazione, il sistema genera dinamicamente un evento compatibile con il calendario nativo del dispositivo (iOS/Android), permettendo all'autista di salvare l'impegno lavorativo con un singolo tocco.
- **Click-to-Chat (WhatsApp API):** Alla conferma di una corsa, si apre automaticamente l'app WhatsApp con un template testuale precompilato (dati passeggero, importo, istruzioni).

Scenario principale di successo

1. L'Admin accede alla pagina `/admin/dashboard`.
2. Il Sistema verifica la sessione admin (Middleware + `checkAdminAuth`).
3. Il Sistema carica le prenotazioni ordinate per data decrescente.
4. L'Admin visualizza la card, che evidenzia i chilometri a vuoto (rimessa) e l'incrocio orari voli/pick-up.
5. L'Admin applica filtri (se necessario).
6. L'Admin seleziona una prenotazione e sceglie un'azione:
    - **Conferma**: Il Sistema imposta stato `confirmed`, logga l'azione e apre WhatsApp.

- **Rifiuta**: L'Admin conferma il dialog, il Sistema imposta `rejected`, logga l'azione e apre WhatsApp.
- **Modifica Prezzo**: Sistema aggiorna `estimated_price`, imposta `price_modified` e logga la modifica.
- **Cancella (Soft)**: Sistema esegue `soft delete` (nasconde dalla dashboard, mantiene in storico).

7. Il Sistema mostra un feedback visivo (toast di successo) e aggiorna la lista.

Estensioni e flussi alternativi

2a. Admin non autenticato *(Passo 2)*
La verifica sessione fallisce. Il Sistema reindirizza l'utente al Login. Le Server Actions restituiscono errore `UNAUTHORIZED`.

3a. Errore caricamento *(Passo 3)*
La query al DB fallisce. Il Sistema mostra un messaggio di errore e un pulsante "Riprova".

5a. Nessun risultato *(Passo 5)*
I filtri non producono risultati. Il Sistema mostra "Nessuna prenotazione trovata".

7a. Azione annullata *(Passo 7)*
L'Admin chiude il dialog di conferma/rifiuto. Nessuna azione viene eseguita sul DB.

7b. Conferma implicita via WhatsApp *(Passo 7)*
Se l'Admin clicca su WhatsApp per una prenotazione `pending`, il Sistema prima tenta di impostare lo stato a `confirmed`. Solo se ha successo, apre la chat.

7c. Vincoli Hard Delete *(Logica Business)*
Tentare un'eliminazione definitiva (`hard delete`) su prenotazioni confermate/completate restituisce errore `INVALID_DELETE_TYPE` per proteggere lo storico.

8a. Errore Polling *(Background)*
Il controllo periodico di nuove prenotazioni fallisce. L'errore viene ignorato silenziosamente per non disturbare l'utente; le notifiche real-time non appaiono finché il servizio non riprende.

9a. Errore aggiornamento DB *(Passo 6-7)*
Fallisce l'update di stato/prezzo o la cancellazione. La Server Action restituisce codice errore (es. `DB_ERROR`). Il client mostra un toast di errore e non aggiorna la vista.

9b. Concorrenza / Non trovato *(Passo 6-7)*
La prenotazione è stata modificata o cancellata da un altro Admin nel frattempo. L'azione fallisce con `NOT_FOUND`. L'Admin deve ricaricare la pagina.

10a. Logout *(Qualsiasi passo)*
L'Admin esegue il logout. La sessione viene distrutta e l'utente reindirizzato al login.

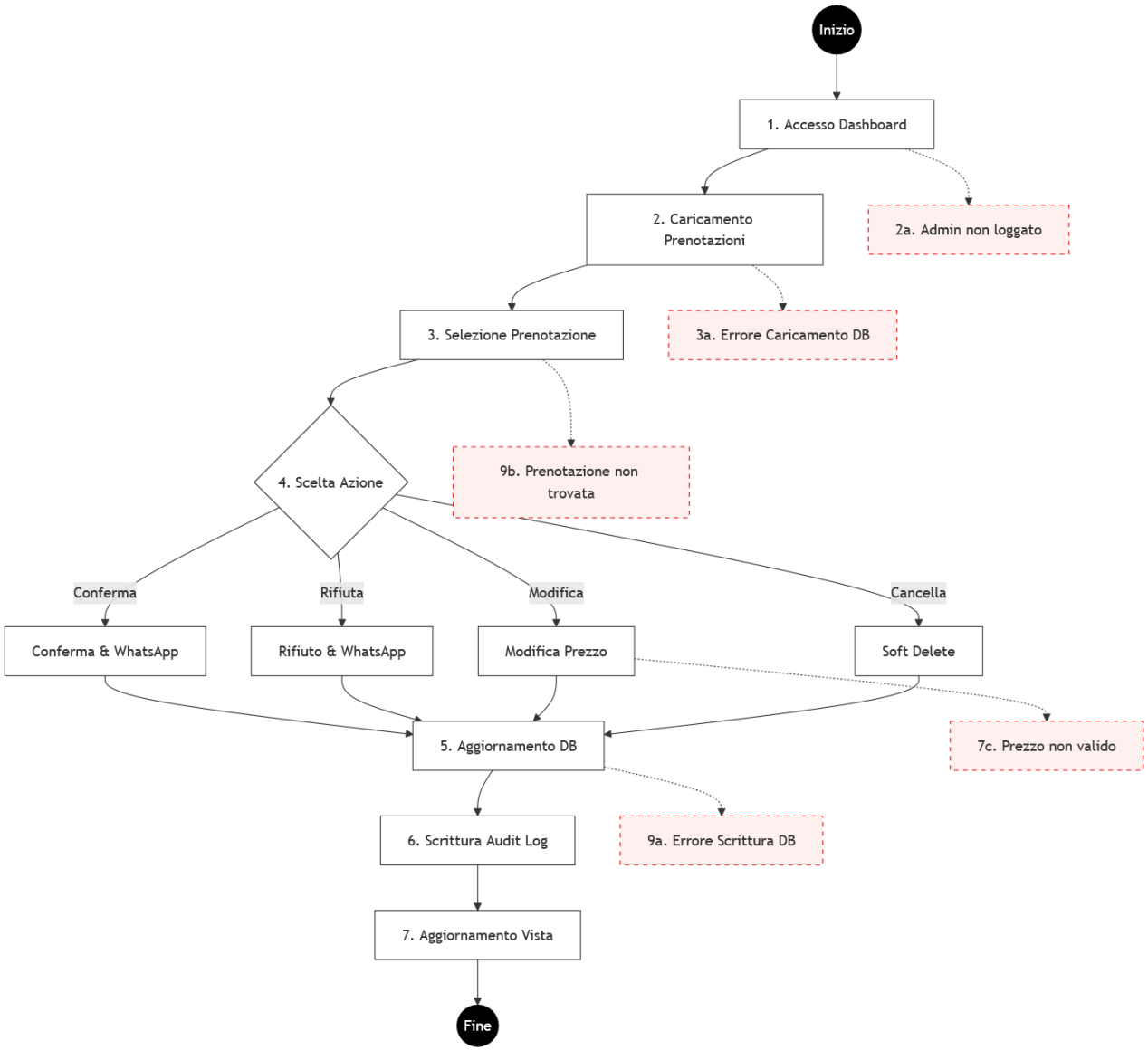

Figura 3: Diagramma di attività - CU2: Gestire Prenotazioni

4.3 CU3: Gestire Preventivi non confermati (Bozze)

Portata Sistema Locridea Transfer

Livello Supporto analitico

Attore primario Sistema (con interazione secondaria del Cliente e dell'Admin)

Parti interessate ed interessi

- **Cliente**: Può interrompere il flusso in qualsiasi momento senza vincoli.
- **Azienda**: Vuole analizzare i tentativi non conclusi per migliorare offerta e UX.
- **Sistema**: Deve registrare le bozze (`booking_drafts`) in modo leggero e renderle visibili.

Scenario principale di successo

1. Il Cliente compila lo **step ricerca** e lo **step dettagli**.
2. Il Cliente compila lo **step anagrafica** e prosegue verso la Privacy.
3. Il Sistema invoca `saveBookingDraft` salvando lo step `privacy` e i dati inseriti.
4. Il Sistema crea il record in `booking_drafts` e restituisce un `draftId`.
5. Il Cliente **abbandona** il flusso.
6. La bozza rimane in stato `submitted = false`.
7. L'Admin accede a `/admin/history` e seleziona il filtro "Non confermate".
8. Il Sistema carica le bozze ordinate per data.
9. L'Admin analizza i dati per fini statistici.
10. L'Admin clicca su "Cancella tutte le bozze" per pulire l'elenco.

Estensioni e flussi alternativi

4a. Cliente conferma la prenotazione *(Successivo al Passo 4)*
Il Cliente completa il flusso (CU1). Il Sistema crea il booking reale e marca la bozza come `submitted = true`. La bozza non appare più tra le "Non confermate".

4b. Salvataggio bozza fallito *(Passo 4)*
Errore tecnico nel salvataggio della bozza. Il Sistema non blocca il Cliente, che può proseguire nel flusso CU1, ma il tentativo non viene tracciato come bozza.

9a. Admin non autenticato *(Passo 9)*
L'accesso alla sezione Storico è negato. Il Sistema reindirizza al Login.

10a. Errore caricamento *(Passo 10)*
La query delle bozze fallisce. L'interfaccia mostra un errore e permette di riprovare.

13b. Errore pulizia manuale *(Passo 12)*
La cancellazione massiva fallisce. Le bozze rimangono visibili e l'Admin riceve un messaggio di errore.

Gestione Privacy *(Nota)*
Le bozze possono contenere dati personali (nome, email, telefono). Questi sono visibili all'Admin per finalità di gestione e analisi, ma devono essere trattati in conformità al GDPR. È prevista la cancellazione manuale per minimizzare la conservazione dei dati.

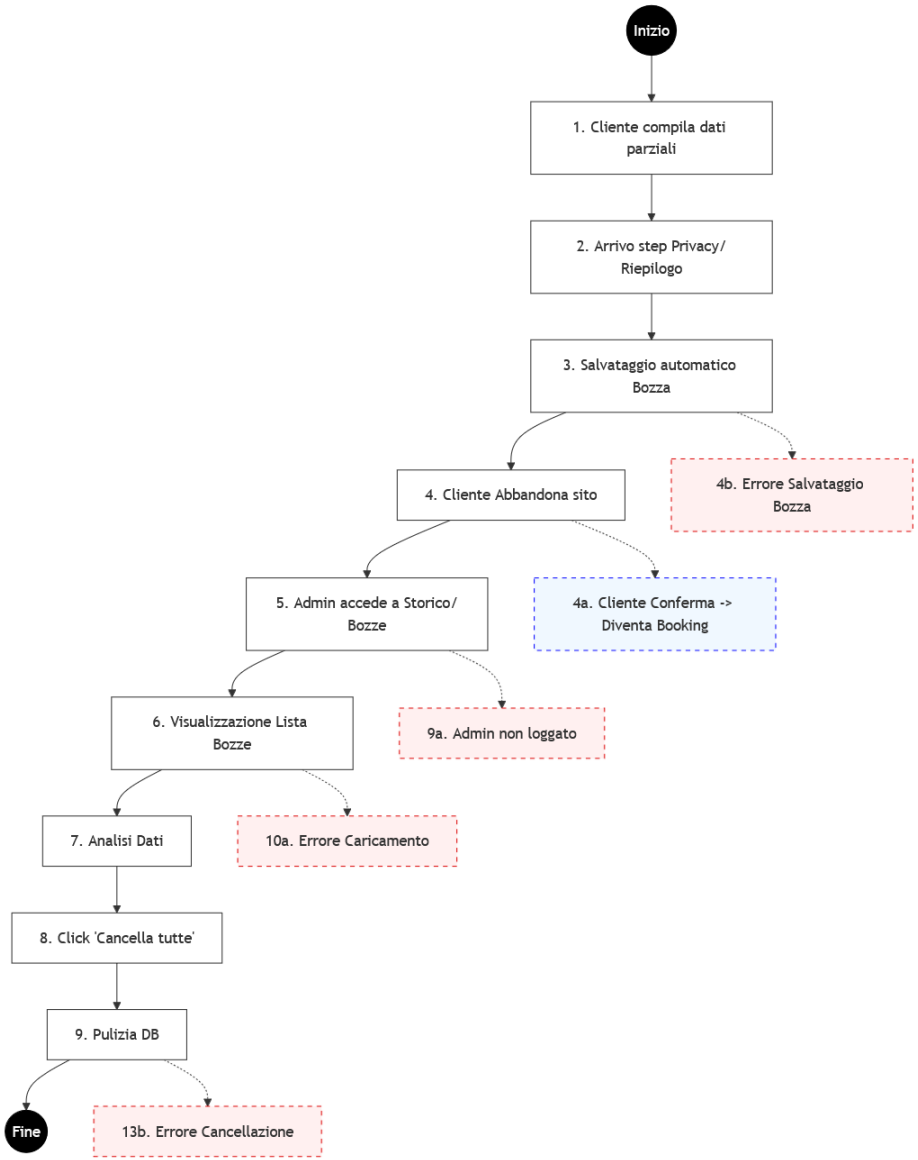

Figura 4: Diagramma di attività - CU3: Gestire Non Prenotate

4.4 CU4: Gestire Tariffe e parametri di calcolo

Portata Sistema Locridea Transfer

Livello Obiettivo utente

Attore primario Amministratore (Admin)

Parti interessate ed interessi

- **Admin**: Vuole modificare i parametri tariffari (prezzo al km, supplementi) in autonomia.
- **Azienda**: Vuole adattare i prezzi a stagionalità mantenendo coerenza.
- **Sistema**: Deve permettere scrittura solo agli Admin, garantendo persistenza.

Scenario principale di successo

1. L'Admin accede alla pagina `/admin/tariffe`.
2. Il Sistema verifica la sessione Admin.
3. Il Sistema carica i parametri correnti (`getPricingSettings`).
4. Il Sistema mostra il form con i valori caricati.
5. L'Admin modifica uno o più campi numerici.
6. Il Sistema valida lato client che i valori siano ≥ 0 e le percentuali ≤ 100.
7. L'Admin clicca su "Salva modifiche".
8. Il Sistema invia i dati a `updatePricingSettings`.
9. Il Sistema verifica nuovamente l'autorizzazione e esegue l'UPDATE sul DB.
10. Il Sistema conferma il salvataggio con messaggio di successo e aggiorna la cache.
11. I nuovi parametri sono attivi per i calcoli futuri.

Estensioni e flussi alternativi

2a. Admin non autenticato *(Passo 2)*
La verifica della sessione fallisce. Il Sistema reindirizza al Login o restituisce errore `UNAUTHORIZED`. Nessuna modifica permessa.

3a. Caricamento fallito *(Passo 3)*
La funzione `getPricingSettings` non riesce a leggere dal DB. Il client mostra un avviso e popola il form con i valori **Predefiniti** (hardcoded nel sistema). L'Admin può comunque salvare per ripristinare la configurazione nel DB.

5a. Valori non validi *(Passo 5-6)*
L'Admin inserisce numeri negativi o percentuali $> 100\%$. Il pulsante "Salva" rimane disabilitato o il sistema mostra errore di validazione.

5b. Ripristino predefiniti *(Passo 5)*
L'Admin clicca su "Ripristina predefiniti". Il form viene compilato con i valori di default del sistema. Le modifiche non sono salvate in DB finché non si clicca "Salva".

9a. Errore scrittura DB *(Passo 9)*
L'aggiornamento su Supabase fallisce (timeout o errore server). Il client mostra un messaggio di errore e mantiene i valori nel form per permettere di riprovare.

9b. Sessione scaduta durante il salvataggio *(Passo 9)*
Tra l'apertura della pagina e il click su Salva la sessione scade. Il sistema rifiuta la modifica e richiede il login.

10a. Uscita senza salvare *(Passo 10)*
L'Admin naviga altrove senza cliccare "Salva". Le modifiche vengono perse e i parametri restano quelli precedenti.

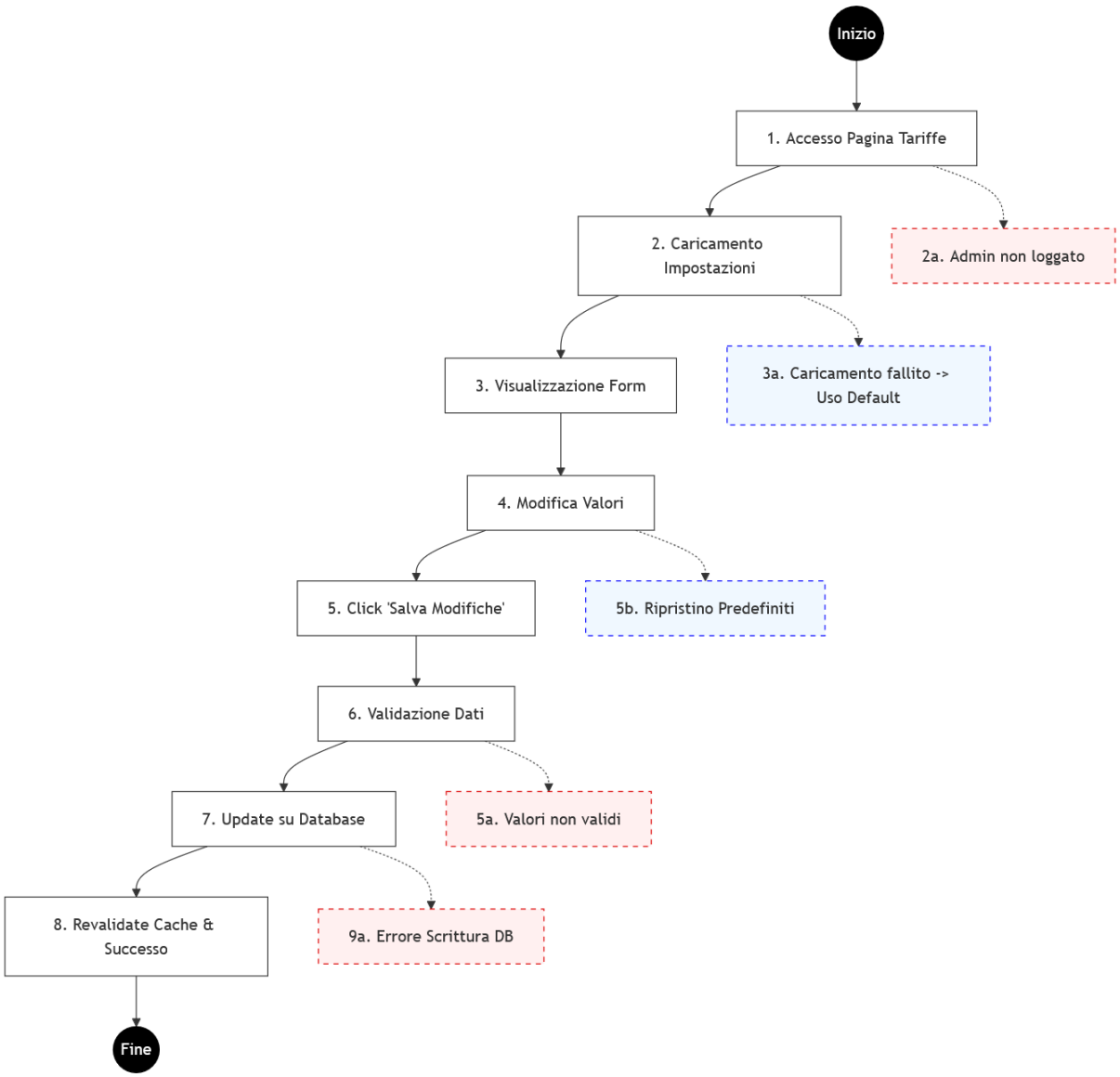

Figura 5: Diagramma di attività - CU4: Gestire Tariffe e parametri calcolo

5 Modello di Dominio

5.1 Descrizione del Modello

Il cuore del sistema è l'entità **Prenotazione (Booking)**, che funge da aggregatore per i dati logistici, anagrafici ed economici calcolati dal motore di pricing. Essa è collegata a:

- Entità **BookingAuditLog** per il tracciamento immutabile degli stati.
- Entità **PricingSettings** che nutre la logica di calcolo.
- Eventuali entità **Package** (Pacchetti Turistici) come via alternativa di instanziazione della prenotazione.

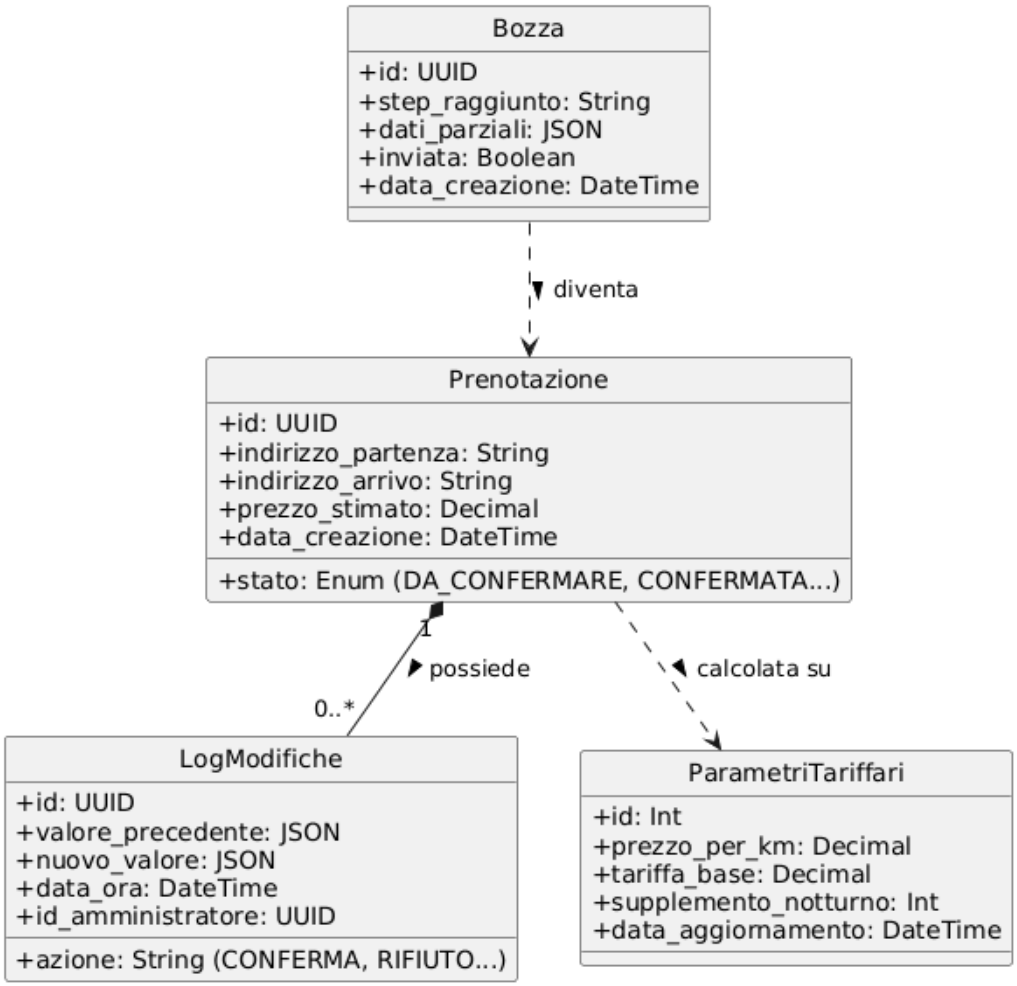

Figura 6: Modello di Dominio del sistema Locridea Transfer

6 Diagrammi di Sequenza del Sistema (SSD)

In questa sezione vengono descritti i flussi di interazione tra gli Attori e il Sistema per i casi d'uso principali.

6.1 SSD CU1: Prenotare Transfer

1. **Cliente** → **Sistema**: Inserisce i criteri di ricerca (Partenza, Arrivo, Data).
2. **Sistema** → **Google Maps API**: Richiede calcolo server-side per evitare manomissioni.
3. **Sistema** → **Pricing Engine**: Il pattern Strategy elabora il costo incluse le variabili di rimessa.
4. **Sistema** → **Cliente**: Mostra preventivo tipizzato.
5. **Cliente** → **Sistema**: Conferma tramite form validato (Zod).
6. **Sistema** → **Database**: Salva la prenotazione in stato `DA CONFERMARE`.

6.2 SSD CU2: Gestire Prenotazioni

1. **Admin** → **Sistema**: Richiede la lista delle prenotazioni in arrivo (Dashboard).
2. **Sistema** → **Database**: Esegue query sulle prenotazioni con stato `pending` (tramite Repository Pattern).
3. **Admin** → **Sistema**: Invia comando di *Conferma*, *Rifiuto* o *Modifica Prezzo*.
4. **Sistema** → **Database**: Aggiorna lo stato e inietta il record in `booking_audit_log`.
5. **Sistema** → **WhatsApp (Esterno)**: Apre la chat con il messaggio precompilato.
6. **Sistema** → **Admin**: Restituisce feedback visivo (toast) e aggiorna la lista.

6.3 SSD CU3: Gestire Preventivi (Bozze)

1. **Cliente** → **Sistema**: Avanza nello step (es. verso Privacy) con dati parziali.
2. **Sistema** → **Database**: Crea o aggiorna il record in `booking_drafts` (senza validazione completa).
3. **Admin** → **Sistema**: Accede alla sezione Storico/Bozze.
4. **Sistema** → **Database**: Recupera le bozze non inviate (`submitted = false`).
5. **Admin** → **Sistema**: Richiede la cancellazione delle bozze.
6. **Sistema** → **Database**: Elimina fisicamente i record delle bozze.

6.4 SSD CU4: Gestire Tariffe

1. **Admin** → **Sistema**: Accede alla pagina Tariffe.
2. **Sistema** → **Database**: Legge la configurazione (`pricing_settings`).
3. **Admin** → **Sistema**: Modifica i valori e salva.
4. **Sistema** → **Sistema**: Valida i dati (es. percentuali ≤ 100, prezzi ≥ 0).
5. **Sistema** → **Database**: Esegue l'UPDATE dei parametri tariffari.
6. **Sistema** → **Admin**: Conferma l'avvenuto aggiornamento e revalida la cache per i calcoli futuri.

7 Architettura Logica e Infrastrutturale

Locridea Transfer supera la concezione classica del pattern MVC implementando un'architettura enterprise robusta, distribuita e fortemente tipizzata, divisa nei seguenti layer:

- **Presentation Layer**: Next.js 16 (App Router), React 19, TailwindCSS. Gestisce il Server-Side Rendering (SSR) e i componenti UI headless.
- **Application Layer (Server Actions)**: Sostituisce le tradizionali API REST fornendo mutazioni dirette e sicure tra frontend e backend. Ingloba l'engine di calcolo logistico e l'integrazione con Google Maps.
- **Data Layer**: Supabase (PostgreSQL), protetto da rigorose policy RLS per garantire che la manipolazione dei record avvenga unicamente tramite contesti autorizzati.

7.1 Design Pattern Implementati

Per gestire l'alta complessità di business e garantire scalabilità e manutenibilità, il core applicativo (directory `@/lib/patterns/`) implementa specifici Design Pattern:

- **Strategy Pattern**: Utilizzato nel modulo `calculateTrip`. Invece di hardcodare le tariffe nel codice, diverse classi Strategy (es. `StandardPricingStrategy`, `FlatPricingStrategy`) si scambiano dinamicamente per calcolare prezzi a scaglioni, supplementi notturni e costi di "rimessa", separando nettamente la logica di business dall'implementazione.
- **Repository Pattern**: Tutte le query a Supabase sono astratte in classi dedicate (es. `BookingRepository`). I componenti e le Server Actions non eseguono mai chiamate dirette al database, agevolando futuri refactoring e testing.
- **Singleton Pattern**: Implementato per la gestione unificata del logging centralizzato in produzione, consentendo la creazione di istanze figlie per tracciare con precisione l'origine degli eventi di sistema.
- **Observer Pattern**: Gestisce il sistema di notifiche UI in modo disaccoppiato, permettendo a componenti non-React (come moduli di utilità) di emettere eventi catturati globalmente dall'interfaccia utente.

7.2 FinOps e CI/CD Automation

L'infrastruttura di distribuzione è stata ingegnerizzata tenendo conto dell'efficienza finanziaria (FinOps). L'hosting su Vercel in contesti multi-team prevede costi di licenza elevati per singolo sviluppatore. Per abbattere radicalmente il TCO (Total Cost of Ownership), è stata progettata una **Ghost CI/CD Pipeline**:

- I trigger nativi Vercel sono stati disattivati per gli account non proprietari.
- Una GitHub Action personalizzata (`deploy.yml`) intercetta i commit sul branch principale.
- Tramite un Webhook crittografato (Deploy Hooks API), GitHub istruisce i server Vercel di avviare il processo di build, garantendo deployment automatizzati a costo zero senza compromettere la sicurezza della pipeline.